documentation (exercise 4)

Roberto Sonzini Gobbi: 12505287

1 Question 1

The implemented clustering algorithms exhibit different behaviors depending on the structure of the pointcloud and the chosen parameters.

Dependence on the number of clusters

Some clustering algorithms, such as k-means and iterative k-means, rely on externally defined parameters that strongly influence their final outcome. In particular, the parameter k, representing the number of clusters, plays a crucial role.

Figure 1 illustrates the effect of an incorrect choice of k using simple 2D examples.

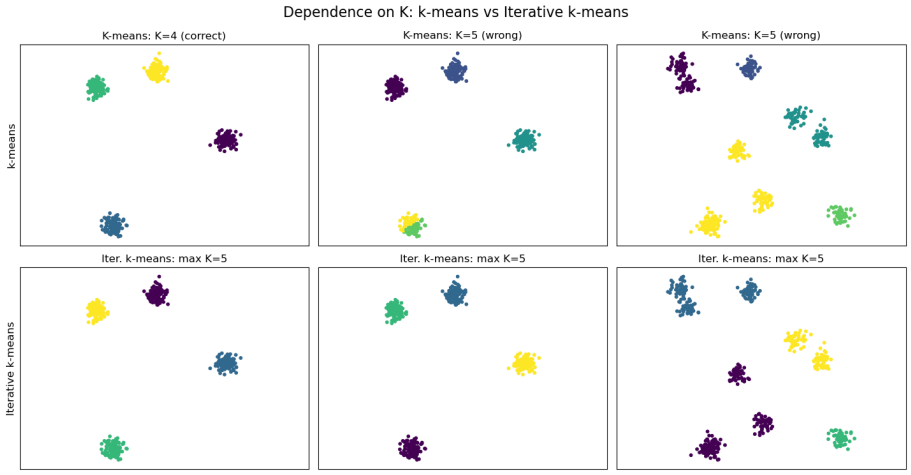


Figure 1: k-dependence 2D examples.

As can be observed, classical k-means fails to correctly separate the data when the true number of clusters is unknown. Iterative k-means partially alleviates this problem by evaluating multiple values of k and selecting the best solution according to a silhouette score. However, this approach still requires defining a maximum number of clusters. If the true number of objects exceeds this upper bound, the algorithm is unable to find the correct clustering.

Furthermore, while classical k-means is computationally efficient, iterative k-means significantly increases the runtime due to the repeated execution of the k-means algorithm for different values of k, as discussed later in the performance analysis.

Figure 2 shows a failure case of k-means on a 3D pointcloud, compared to the result obtained with iterative k-means.

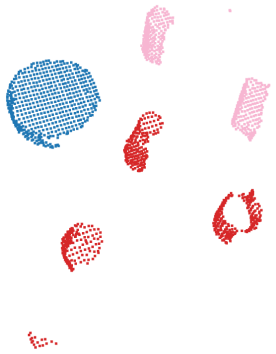


Figure 2: k-means with $k = 3$



Figure 3: iterative k-means

Sensitivity to outliers and noise

Outliers and noise play a crucial role in clustering, and the implemented algorithms handle them very differently. K-means and iterative k-means do not provide any explicit mechanism for outlier detection; consequently, outliers can significantly influence centroid positions and degrade clustering quality.

Since k-means enforces a fixed number of clusters, even a small number of outliers may shift centroids enough to prevent correct object separation.

DBSCAN, in contrast, explicitly identifies outliers by assigning them the label -1. However, if outliers are spatially close to each other or located between clusters, they may form density bridges. Depending on the chosen parameters, this can lead to the undesired merging of distinct clusters.

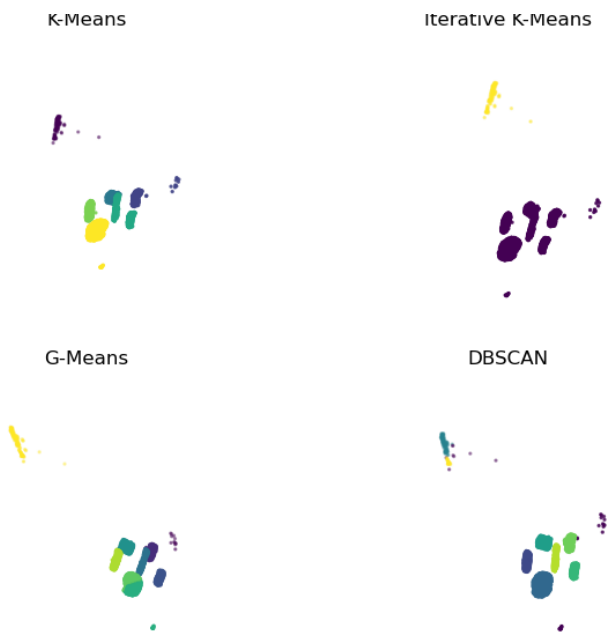


Figure 4: Outliers sensibility.

To further illustrate this effect, Figure 5 shows the behavior of the algorithms on synthetic 2D data.

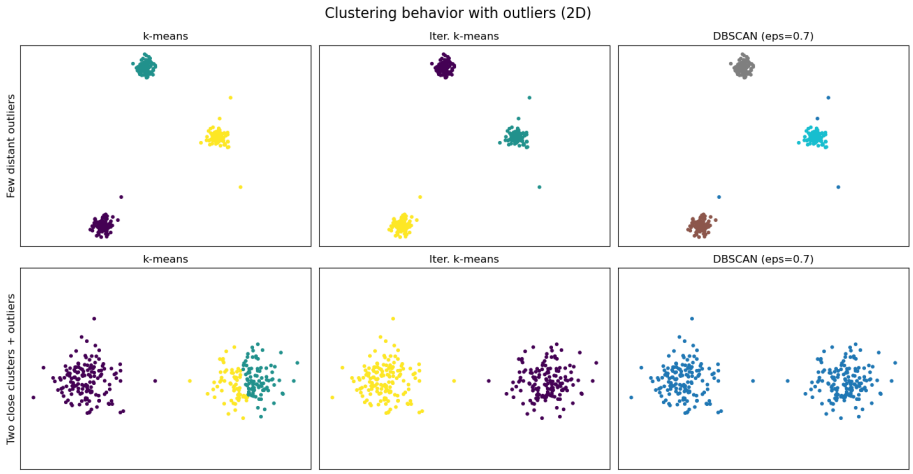


Figure 5: Outliers sensibility in 2D.

In this example, DBSCAN successfully identifies outliers in the first case. However, depending on the choice of the parameter ϵ, it may incorrectly merge nearby clusters as happens in he second case shown. In a 3D scene, this behavior is particularly problematic, as it can result in treating multiple objects as a single one.

Close objects

Objects that are spatially close to each other represent a particularly challenging scenario for clustering algorithms. For DBSCAN, the parameter ϵ plays a critical role. If ϵ is chosen too large, points belonging to neighboring objects may fall within the same density neighborhood, causing distinct objects to be merged into a single cluster. Although adjusting ϵ can resolve this issue in some cases, an optimal value for ϵ a priori is often difficult to determine, especially for scenes with varying object sizes and densities.

K-means can also fail in this situation, as shown in the figure, because it relies on minimizing intra-cluster variance and assumes that clusters are well separated and roughly spherical. When objects are close to each other, this assumption is violated, leading to incorrect cluster assignments.

G-means may also produce incorrect results in this scenario. When two objects are spatially close, the combined set of points may still pass the Gaussianity test along the principal projection axis. As a result, the algorithm may fail to split the cluster, even though it actually contains multiple distinct objects.



Figure 6: Close objects problem

Stacked objects

When objects are stacked on top of each other, clustering becomes particularly difficult. Since points from different objects are spatially close, many algorithms tend to group them into a single cluster.

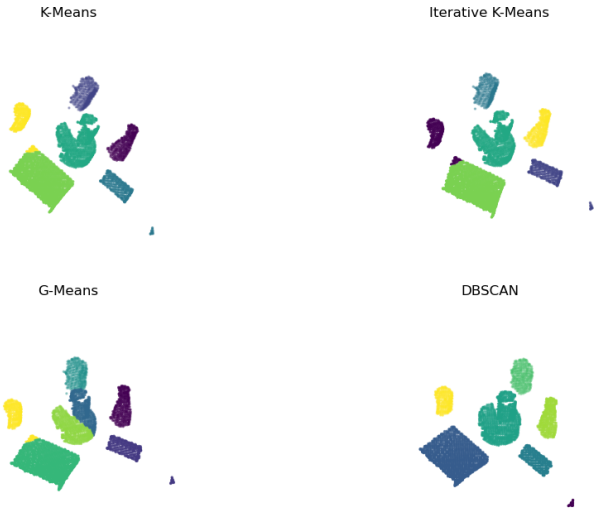


Figure 7: stacked objects.

In this scenario, none of the algorithms perfectly separates the stacked objects. Classical k-means fails despite using the correct number of clusters, as the spatial separation between the objects is insufficient for variance-based clustering. DBSCAN also merges the objects due to the high local point density. G-means is the only algorithm that attempts to split the cluster, although the resulting separation is still not fully correct.

Overall performances

To evaluate the practical usability of the clustering algorithms, I've also compared their execution times. The measured runtimes for multiple pointclouds are summarized in Table 1 and visualized in Figure 8.

Execution times

PointCloud	K-Means	Iterative K-Means	G-means	DBSCAN
0	0.1379	2.8072	1.0407	0.0891
1	0.1675	2.6882	1.0521	0.0758
2	0.1545	2.4669	0.9268	0.0696
3	0.1199	2.9779	1.1166	0.0796
4	0.1231	2.4752	0.9864	0.0741
5	0.2186	3.0235	0.9221	0.0791
6	0.2251	3.5848	0.9388	0.0871
7	0.2128	2.9208	1.0117	0.0828
8	0.2352	3.1439	0.8429	0.0746
9	0.1971	4.5215	1.0271	0.1303
Average	0.1792	3.0610	0.9865	0.0842

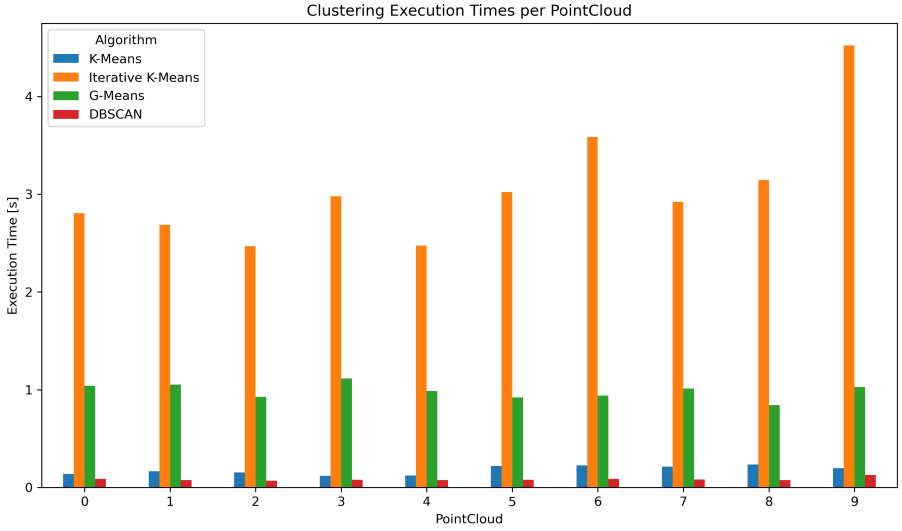


Figure 8: timing graph

As expected, iterative k-means is the most computationally expensive algorithm due to the repeated execution of k-means for different cluster counts. Classical k-means and DBSCAN achieve very low runtimes but require careful parameter tuning. G-means offers a good compromise between adaptability and computational cost, although it remains slower than k-means and DBSCAN due to the statistical test that must be performed.

Overall, the experiments demonstrate that no single clustering algorithm performs optimally in all scenarios. The choice of method strongly depends on the structure of the pointcloud, the presence of noise, and the required level of automation.

Digression on cluster shapes

Another key difference between the clustering algorithms is the assumed cluster shape. K-means assumes roughly spherical clusters with similar variance, making it unsuitable for elongated or irregular shapes. DBSCAN, being density-based, can handle arbitrary shapes if the density parameters are chosen appropriately. G-means assumes Gaussianity along the principal direction and may fail when clusters overlap or deviate from a Gaussian distribution. While illustrated on simple 2D examples, similar effects are observed in the 3D pointcloud experiments.

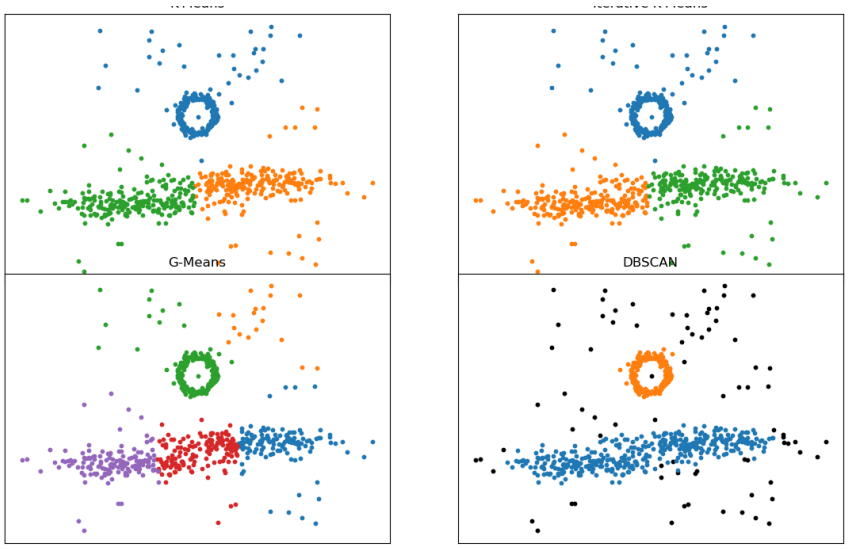


Figure 9: behaviour on different cluster shapes

(Further experiments on synthetic 2D datasets could highlight additional behaviors, however, this report concentrates on the cases observed in the provided 3D pointclouds, which more accurately represent realistic scenarios.)

Conclusion

In summary, k-means is fast and simple but requires prior knowledge of the number of clusters and is highly sensitive to outliers. Iterative k-means reduces the dependency on a fixed k at the cost of significantly increased runtime. G-means provides a more adaptive approach by estimating the number of clusters automatically, but relies on statistical assumptions that may not hold in complex 3D scenes. DBSCAN is robust to noise and does not require specifying the number of clusters, but its performance strongly depends on the choice of density parameters and struggles with objects of varying densities.

2 Question 2

The effect of downsampling on the runtime of the implemented clustering algorithms was evaluated on two pointclouds at two different voxel sizes. As expected, all algorithms benefit from reducing the number of points, since fewer points need to be processed at each step.

Classical k-means shows a significant improvement in runtime, as most of its operations scale linearly with the number of points. Iterative k-means remains the slowest algorithm due to the repeated execution of k-means for multiple cluster counts.

G-means also benefits from downsampling, although a portion of its computational complexity is intrinsic to the algorithm, namely the projection of points and the statistical normality tests.

DBSCAN generally improves as well, because its runtime depends on the number of points. However, in regions of high density, the need to check all neighboring points can limit the improvement.

It is important to note that downsampling can also affect the quality of clustering results. Reducing the number of points may decrease the density of individual objects, making clusters harder to identify, especially for density-based algorithms like DBSCAN. Outliers can have a stronger influence, and statistical tests in algorithms like G-means may produce different results. The overall result depends on the downsampling intensity and quality.

PointCloud	Downsampling	K-Means	Iterative K-Means	G-Means	DBSCAN
PC0	voxel_0.01	0.1295	2.4392	0.9316	0.0735
PC0	voxel_0.02	0.0406	0.7844	0.2756	0.0106
PC1	voxel_0.01	0.1997	5.0744	1.0923	0.1398
PC1	voxel_0.02	0.0774	1.4506	0.4429	0.0175

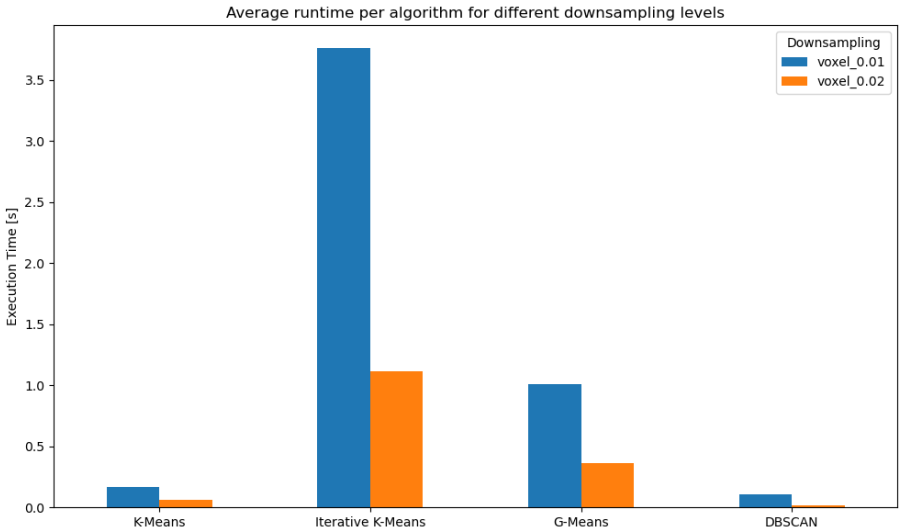


Figure 10: downsampling effect on timings

The comparison was also conducted using two different downsampling methods, highlighting the same performance behaviors in terms of runtime and supporting the observations made earlier. The general behaviour is in fact the same. We see different times because the two downsampling algorithms works in a different way and so the effective number of points could be different causing different runtimes.

Table 1: Average runtimes [s] for different downsampling levels

Downsampling	K-Means	Iterative K-Means	G-Means	DBSCAN
voxel_0.01	0.1808	3.5652	1.0158	0.0959
voxel_0.02	0.0504	0.9702	0.3296	0.0118
uniform_k5	0.7789	24.0713	2.3222	1.4533
uniform_k10	0.3477	8.3766	1.4481	0.3977

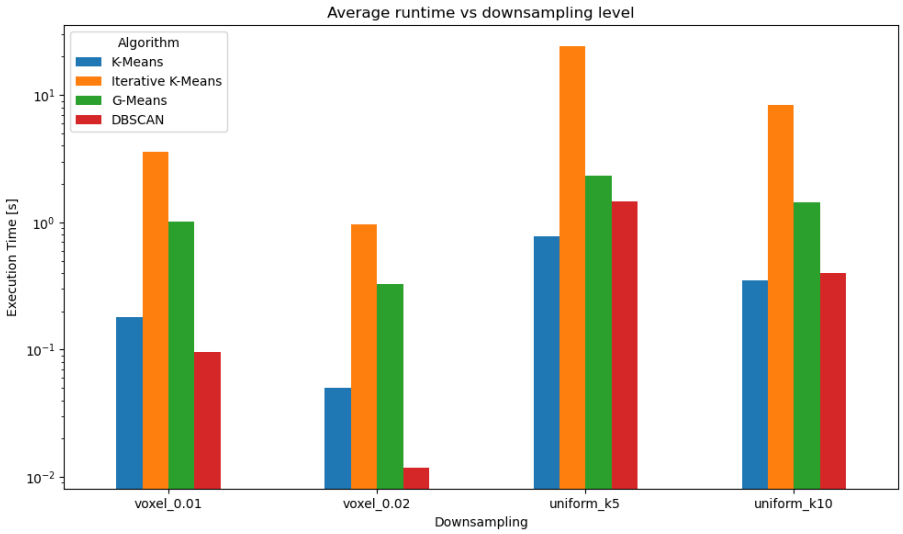


Figure 11: downsampling algorithm comparison on timings log scale

Overall, the experiments confirm that downsampling effectively reduces computation times for all algorithms. The relative improvement depends on the algorithm's intrinsic complexity as explained above.

The results shown in the plots are the averages over two different point clouds to illustrate the general behavior of the downsampling.

3 Question 3

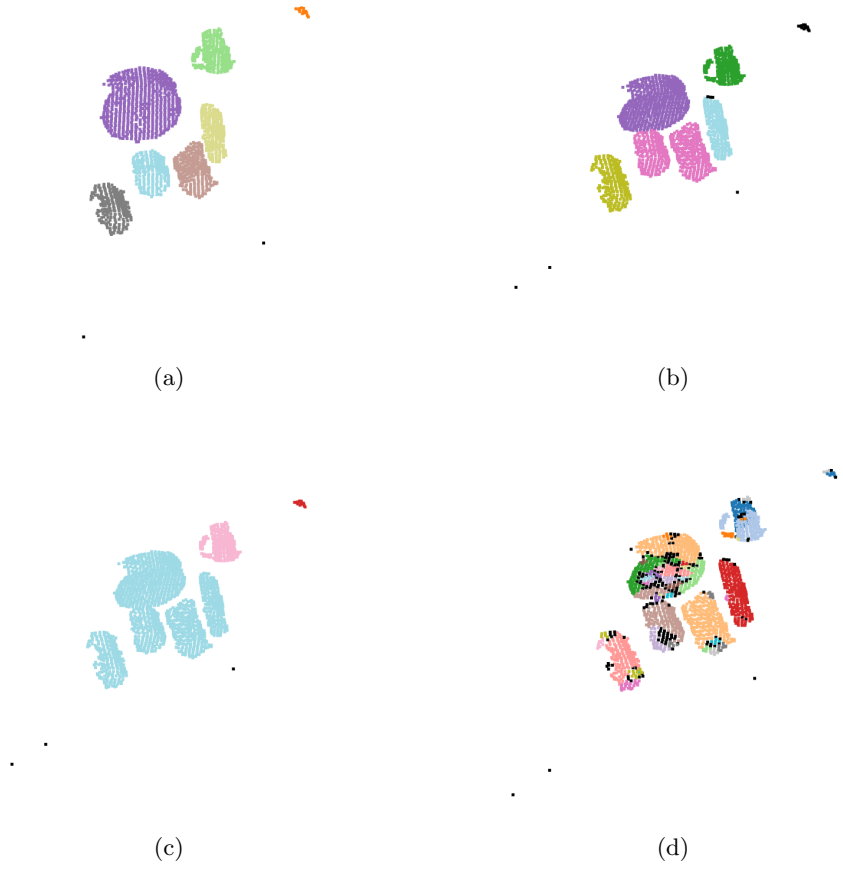
DBSCAN is a density-based clustering algorithm controlled by two parameters: the neighborhood radius ϵ and the minimum number of points min_samples required to form a dense region. These determine whether a point is classified as a core point, border point, or noise, strongly influencing the clustering result.

The parameter ϵ defines the maximum distance between two points for them to be considered neighbors. If ϵ is chosen too small, only very close points are considered density-reachable, which often leads to fragmented clusters and a large number of points being classified as noise. This effect is especially visible in pointclouds containing sparse regions or thin object structures. Conversely, if ϵ is chosen too large, neighborhoods may include points from different objects, causing distinct clusters to merge and reducing the segmentation quality.

The parameter min_samples specifies the minimum number of points within an eps-neighborhood required for a point to be considered a core point. A small value of min_samples makes DBSCAN sensitive to noise, as even small random point groups may form clusters. A large value increases robustness against noise but may lead to small or less dense objects being entirely classified as noise.

Both parameters are strongly interdependent and must be tuned together. Increasing eps typically requires increasing min_samples to avoid merging clusters, while decreasing ϵ often requires reducing min_samples to prevent excessive noise classification. In practice, the choice of these parameters depends heavily on the point density of the dataset.

Hereafter, several images are presented to illustrate the behaviors discussed above. In particular, we can observe how a too high value of min_samples, combined with a medium value of ϵ, leads to some points being classified as noise. We also show how a large ϵ can cause the merging of multiple objects into a single cluster (Figure 12c). Additionally, a case with correct clustering achieved through careful manual fine-tuning of the parameters is presented (Figure 12a), and finally a scenario where both min_samples and ϵ are set too low, resulting in poor clustering performance (Figure 12d).



(a) (b)

(c) (d)

Figure 12: Results for different parameters